The page object needs a `getFeaturedDestinations()` method that returns a list of `DestinationDeals`. Internally, this method needs to convert the data in the featured destinations list into a list of `DestinationDeals`.

Let's walk through how you might implement this method. To start, you could use the `@FindBy` annotation to retrieve the list of featured destination `<div>` blocks:

```
@FindBy(css = ".featured .featured-destination")
private List<WebElement> featuredDestinations;
```

Next, you'd write the `getFeaturedDestinations()` method itself:

```
public List<DestinationDeal> getFeaturedDestinations() {
    List<DestinationDeal> deals = Lists.newArrayList();
    for(WebElement destinationEntry : featuredDestinations) {
        deals.add(destinationDealFrom(destinationEntry));
    }
    return deals;
}
```

Populate the list by converting the web elements to Destination-Deals.

Return a list of DestinationDeals.

Create a new empty list.

The code here loops through the web elements representing the featured destination `<div>` blocks and converts each to a `DestinatonDeal` object. The details of the conversion process are left to the `destinationDealFrom()` method, shown here:

```
private DestinationDeal destinationDealFrom(WebElement destinationEntry) {
  String destinationCity = destinationEntry.findElement(
                           By.className("destination-title")).getText();
  String priceValue = destinationEntry.findElement(
                           By.className("destination-price")).getText();
  int price = Integer.parseInt(priceValue.substring(1));
  return new DestinationDeal(destinationCity, price);
}
```

Retrieve the destination title. ❶

Retrieve the destination price. ❷

Convert the price to an integer.

Create a new DestinationDeal using these values. ❸

This method reads the title ❶ and price ❷ from the web element and uses them to build a new `DestinationDeal` ❸.

PAGE OBJECTS SHOULD REPORT ON PAGE STATE

Page objects are responsible for reporting information about the contents or state of the page (or page component) to the test. The test can then use this information to perform any required checks and assertions; page objects shouldn't contain assertion logic.

For example, on the flight booking page in figure 8.13, there's a Search button. Suppose that this button is disabled until all of the required fields are filled. To test this, you could add a method with an embedded assertion to the corresponding page object, as shown here:

```
public class BookingPage {
    ...
    WebElement search;

    public void searchButtonShouldBeEnabled() {
        assertThat(search.isEnabled(), is(true));
    }
}
```

Represents the Search button.

This will fail if the Search button isn't enabled.